INSIDE

medit

Building a Terminal-Based Code Editor
from Scratch with Java

A source-grounded engineering case study of architecture, data structures, rendering, language tooling, performance and the developer journey.

Mohammed Elmaghraby

first edition

Technical edition • 31 October 2026

About this edition

This book was prepared from the uploaded maghraby-editor source archive. It describes the code that was present in that archive, not an imagined idealized editor. The source tree includes Java implementation files, tests, Maven configuration and a README. The archive also contains IDE and Git metadata; generated build output was not treated as source.

The README labels the project version 3.0 in its run instructions and describes a Piece Table v2 engine, while the uploaded pom.xml declares Maven artifact version 4.0. Those are different version labels in different project surfaces; this book does not assume they are synchronized.

Verification boundary: this environment did not have Maven installed, so the test suite could not be executed here. The analysis is a source review, not a successful build/test certification. All proposed fixes are presented as review recommendations and have not been applied to the uploaded ZIP.

How to read the code examples

Listings are deliberately small extracts or replacement patterns. They explain the relevant algorithm without reproducing every source file. Where a patch is proposed, compare it with the exact current method before applying it, then add regression tests and run the project on the target terminal.

Contents

Part I — The project and its shape

01. The idea: an editor, not just a wrapper
02. Reading the repository and separating claims from implementation
03. Runtime architecture and thread boundaries

Part II — The editing engine

04. The Piece Table
05. The line index and lazy delta
06. Undo and redo as edit deltas
07. Cursor coordinates, Unicode and editing semantics
08. File persistence and the save path

Part III — The user interface

09. Terminal ownership and ANSI rendering
10. Input decoding and modal editing
11. Rendering, invalidation and syntax highlighting
12. File tree, panes and document state

Part IV — Language tooling

13. Syntax highlighters and state caching
14. Autocomplete and Language Server Protocol
15. Process execution and compiler planning

Part V — Engineering review

16. Performance: what the data structures really cost
17. Memory, resource lifetimes and failure handling
18. Concrete fixes and regression tests
19. Benchmarking and measurement

Part VI — The developer's journey

20. The challenges behind medit

21. A practical roadmap
Appendices — Commands, architecture map and review checklist

CHAPTER 01

The idea: an editor, not just a wrapper

> A terminal editor becomes a serious engineering project when it owns its buffer model, input loop, renderer and process lifecycle.

medit is a modal terminal text editor written in Java. It uses a real terminal rather than a graphical window, and it implements its own editing core. That distinction matters: the project is not merely a Java front end around an external editor. The source contains a custom text buffer, edit history, cursor state, input decoding, ANSI output, file navigation, language tooling and a code runner.

The project's practical promise is an integrated loop: open a source file, edit it, receive language-aware assistance, save it, compile or run it, then return to the editor without losing the terminal session. This makes the editor both a text manipulation system and a small development environment.

The product boundary

The editor owns document text and user interaction. External tools remain responsible for compilation and execution, and language servers provide richer semantic information. This is a sensible boundary: medit does not need to reimplement javac, GCC, Python, or the semantics of every programming language.

Why the project is technically meaningful

• A custom Piece Table (thanks samouly) and line index require careful offset invariants.

• Undo/redo must preserve text and cursor semantics across line breaks.

• A terminal is a shared stateful device: raw mode, alternate screen and child processes must be coordinated.

• Rendering and language-server callbacks cross thread boundaries.

• A code runner must preserve user files, terminal usability and process cleanup.

Engineering note: The strongest portfolio story is not the number of features. It is the way the features meet at well-defined interfaces.

CHAPTER 02

Reading the repository and separating claims from implementation

The source tree is organized by responsibility. The package structure provides a useful map of the system:

Package / file group	Responsibility
org.maghraby.Main	Application startup and shutdown orchestration
core: TextEditor, Doc, Line, State, Mode	Document model, shared editor state and editing modes
ds: Node, MyStack, EditAction	Custom linked stack and undo/redo records
input: Input, Commands, Autocomplete, EditListener	Keyboard decoding, commands, suggestions and mutation notifications
ui: Renderer, Terminal, Ansi, Panes, Sidebar, FileIcons	Terminal control, screen drawing, splits and navigation
syntax: highlighters, registry, cache	Language-specific lexical coloring and multiline state
lsp: LspClient, LspDocumentSync, LspManager, LspServers	JSON-RPC process, document synchronization and server selection
runner: CodeRunner	Compiler planning, execution and temporary build cleanup
util: Json	JSON serialization and parsing
test tree	JUnit tests and JMH benchmark class

A crucial accuracy distinction

The README describes some operations as O(1), zero-copy or zero-allocation. Source inspection shows the implementation is more nuanced. Piece lookup scans the piece list; Java ArrayList insertion/removal can shift later pieces; the line index sometimes moves a pending delta across a range and shifts line-start entries; and saving ADD pieces currently calls StringBuilder.substring, creating temporary Strings. The architecture is performance-conscious, but its complexity claims should be stated with these costs included.

Similarly, the README says 'zero-dependency' and 'pure Java'. The runtime avoids a third-party terminal UI framework and uses Java plus POSIX terminal commands, but the Maven project does declare JUnit and JMH test-scope dependencies. 'No runtime UI framework dependency' is a more precise claim.

Engineering note: A good technical book should not repeat marketing claims without checking the code path that implements them.

CHAPTER 03

Runtime architecture and thread boundaries

Main.main first captures the terminal's original stty state. If it cannot inspect a real terminal, startup stops with a clear error. When a filename is supplied, the first document is loaded and LSP startup is requested. The application then enters the alternate screen, starts the render thread and resize watcher, and runs the input loop on the main thread.

CODE LISTING

```
main(args)
  capture original terminal mode
  optionally load initial document + start LSP
  register shutdown hook
  update terminal dimensions
  enter alternate screen / raw mode
  start render thread
  start resize watcher
  run input loop
  finally:
    stop application loop
    join renderer briefly
    restore terminal
    stop language servers
```

Three important execution contexts

Context	Typical work	Design constraint
Input/main thread	Read bytes, decode keys, invoke edit commands	Must remain responsive; avoid slow I/O
Render thread	Read editor state, draw visible lines and status	Must not render while a child process owns the terminal
Background workers	LSP reader/sync, resize watcher	Callbacks must be thread-safe and must not block UI

The Terminal class exposes a render lock and a renderPaused flag. The code runner suspends the editor screen before launching a child process, then restores raw mode and redraws after the process exits. The resize watcher attempts to use the UNIX WINCH signal and retains a two-second polling fallback.

Engineering note: The most important concurrency rule is ownership: only one actor should write a coherent terminal frame at a time. The source already uses a render lock around terminal handover; every new UI or background feature should respect that boundary.

CHAPTER 04

The Piece Table

TextEditor is built around two text stores and a sequence of pieces. originalBuffer holds the loaded text and is treated as read-only. addBuffer accumulates inserted text. Each Piece identifies a source buffer, a start offset and a length. The visible document is the concatenation of those referenced slices in piece-list order.

CODE LISTING

```
Piece {
    source: ORIGINAL or ADD
    start: offset inside the chosen backing buffer
    length: number of characters
}

Document = piece[0] + piece[1] + ... + piece[n-1]
```

Insertion

An insertion at document offset k scans the pieces while maintaining the cumulative document position. It then inserts a new ADD piece at a boundary, or splits an existing piece into left and right slices and places the new piece between them. When new text is appended immediately after the previous ADD slice in addBuffer, the implementation coalesces the slices by extending the previous piece.

Deletion

Deletion walks the pieces intersecting the requested range. An overlapped piece can be trimmed at the tail, trimmed at the head, split into two surviving pieces, or removed completely. The algorithm preserves the backing text and changes only the piece descriptors. A special releaseAddTail method truncates the end of addBuffer when a removed ADD piece is also the last referenced add-buffer segment.

Real complexity

Operation	Cost in this implementation
Find insertion point	O(P) scan over P pieces
Insert piece into ArrayList	O(P) worst case due to element shifting
Delete range	O(P + K) in broad terms, where K is affected pieces; list removals may shift entries
Read a range	Depends on pieces traversed plus characters copied
Save	O(N) output, plus allocation from ADD substring calls in current code

The Piece Table avoids copying the entire document for every local edit. That is its main win. It does not make every edit mathematically constant-time: descriptor lookup and list maintenance remain costs. Coalescing is important because it keeps P from growing by one for every typed character in common sequential typing.

Core invariant

> The ordered pieces must describe exactly docLength characters, and every referenced slice must remain within its backing buffer. No two live pieces should accidentally overlap the same logical document range.

Whenever insertText or deleteText changes pieces, docLength and the line index must be updated in the same synchronized operation. If any one of these falls out of sync, cursor mapping, rendering and saving can all fail in different ways.

CHAPTER 05

The line index and lazy delta

A terminal editor needs frequent conversions between a (row, column) cursor and a linear document offset. TextEditor maintains lineStarts: the offset at which each line begins. Line zero starts at offset zero; each newline contributes a start position immediately after itself.

The line index is updated incrementally. For an edit, upperBound performs a binary search for the first line start greater than a given offset. indexInsert counts newlines in inserted text, shifts later line starts, inserts starts for new lines and updates a pending delta. indexDelete removes starts inside the deleted range and adjusts the remaining offsets.

The delta / deltaFrom representation

Entries at or after deltaFrom are interpreted with a pending delta added by ls(i). This allows a run of later line starts to share one deferred offset adjustment rather than eagerly rewriting every entry for every character typed on a line. moveDeltaTo materializes the pending adjustment when the edit point moves across the boundary.

CODE LISTING

```
private int ls(int i) {
    return i >= deltaFrom ? lineStarts[i] + delta : lineStarts[i];
}

private void moveDeltaTo(int idx) {
    if (deltaFrom > lineCount) deltaFrom = lineCount;
    if (idx < deltaFrom) {
        for (int k = idx; k < deltaFrom; k++) lineStarts[k] -= delta;
    } else {
        for (int k = deltaFrom; k < idx; k++) lineStarts[k] += delta;
    }
    deltaFrom = idx;
    if (deltaFrom >= lineCount) delta = 0;
}
```

This is a clever deferred-update technique, but it is not a universal O(1) line-index update. moveDeltaTo may loop over a range, and inserting or removing line starts uses System.arraycopy. Character-only edits often take the cheap path; edits that change line structure can require work proportional to the number of shifted line starts.

What must be tested

- Insert a normal character at the start, middle and end of a line.
- Insert multiple newline characters in one operation.
- Delete across one or many line boundaries.
- Delete the final newline and the first newline.
- Undo and redo edits that split and merge lines.
- Run the internal line-index consistency assertion after randomized edit sequences.

Engineering note: The most valuable test is a property test: compare the incremental index against a full scan after every random mutation.

CHAPTER 06

Undo and redo as edit deltas

The history system stores EditAction records rather than complete document snapshots. An action contains an operation type (INSERT or DELETE), a row, a column, the affected text and a forward flag used to preserve Delete-key cursor behavior.

MyStack is a custom singly linked stack. Node stores an EditAction and a next pointer. push adds at the head, pop removes the head, and clear drops the chain. At 1,500 entries, push calls removeBottom to discard the oldest action, bounding history depth.

CODE LISTING

```
public void push(EditAction data) {
    if (size >= MAX_CAPACITY) removeBottom();
    Node n = new Node(data);
    n.next = top;
    top = n;
    size++;
}
```

Undo / redo state transition

For an INSERT action, undo deletes the inserted text at its recorded position; redo inserts it again. For a DELETE action, undo restores the removed text; redo removes it again. The two stacks exchange the action as the operation is replayed. A new user edit clears redo history.

Trade-offs

Delta history is substantially more memory-efficient than full-document snapshots when edits are small relative to the document. The cost is correctness complexity: action coordinates must still identify the intended text after later actions have been undone in reverse order. Grouping keystrokes into logical edits could improve usability, but requires a policy for time gaps, cursor movement, newlines and mode changes.

The fixed limit bounds the number of actions, not the total number of characters stored. A single paste action can contain a large String. A stricter memory budget would track both action count and retained text length.

Engineering note: Potential enhancement: implement a byte/character budget in addition to MAX_CAPACITY, while keeping the linked stack's oldest-entry eviction policy.

CHAPTER 07

Cursor coordinates, Unicode and editing semantics

The editor stores a cursor as row and column coordinates. TextEditor maps this to an absolute offset using ls(row) + col. Conversely, upperBound(offset) identifies the line containing an offset. This mapping underpins movement, insertion, deletion, undo/redo and LSP synchronization.

Java String indexes are UTF-16 code-unit indexes, not terminal display columns. The source contains explicit surrogate-pair handling for backward and forward deletion: if the cursor is between a high and low surrogate, both code units are removed together. That is a meaningful protection against splitting many supplementary Unicode characters.

The remaining Unicode distinction

A Unicode code point can occupy one or two UTF-16 units, while a grapheme cluster may contain multiple code points (for example a base letter plus combining marks or an emoji joined with a zero-width joiner). Terminal display width is yet another measure: some glyphs occupy two columns. Therefore a logical buffer column is not always the same as a terminal screen column.

LSP positions in this code are explicitly documented as UTF-16 units, which aligns with Java String indexing and the LSP default encoding model. The renderer, however, must translate text into terminal cells carefully if it aims to align cursor placement for wide or combining characters.

Recommended cursor model

- Keep document offsets in UTF-16 units for compatibility with Java Strings and LSP positions.
- Introduce a separate display-column function for terminal layout.
- Test tabs, CJK wide characters, combining marks, emoji and surrogate pairs.
- Clamp cursor columns against logical line length after edits and file switches.

CHAPTER 08

File persistence and the save path

TextEditor.saveToFile opens a UTF-8 BufferedWriter and streams the piece sequence in order. ORIGINAL pieces are written directly from originalBuffer using Writer.write(String, offset, length). ADD pieces currently use addBuffer.substring(start, end), which creates a temporary String for each ADD piece.

The save method also writes one trailing newline when the document is non-empty and does not already end in newline. This is a policy choice; it means the in-memory text can differ from the bytes written to disk. The dirty flag is then cleared. That policy should be explicit and tested because a reload may produce a final newline that was not present in the current buffer.

Recommended zero-intermediate-String change

CODE LISTING

```
for (Piece p : pieces) {
    if (p.source == BufferType.ORIGINAL) {
        writer.write(originalBuffer, p.start, p.length);
    } else {
        // Writer implements append(CharSequence, start, end).
        // StringBuilder is a CharSequence; no substring String is needed.
        writer.append(addBuffer, p.start, p.start + p.length);
    }
}
```

This avoids the explicit substring allocation for each ADD piece. It does not make the complete save operation allocation-free: the writer has buffers, the JVM may perform internal work, and filesystem I/O still has costs. It does make the implementation match the intended streaming design more closely.

Safer persistence

For crash resilience, consider writing to a sibling temporary file, flushing and closing it, then replacing the destination with an atomic move when supported. Preserve permissions where appropriate and handle failures without marking the document clean. That design avoids truncating the original file before a failed write completes.

CHAPTER 09

Terminal ownership and ANSI rendering

Terminal is a low-level adapter. It captures and restores stty settings, enters raw/no-echo mode, switches to the alternate screen, hides and shows the cursor, reads terminal dimensions and temporarily yields the terminal to a child process.

The editor's terminal session is stateful. If the process exits while raw mode is active, the user's shell can appear broken: typed characters may not echo, line editing may fail, or the cursor may remain hidden. Main therefore registers a shutdown hook and calls restoreTerminal in both exception and normal-finally paths. An AtomicBoolean makes restoration idempotent.

The child-process handover

CODE LISTING

```
suspendTerminal()
try {
    summary = CodeRunner.execute(plan, out)
    show "Press Enter to return"
    waitForEnter()
} finally {
    CodeRunner.cleanup(plan)
    resumeTerminal()
}
```

The user's earlier terminal-output issue was addressed by restoring terminal state with stty sane in the execution workflow. The current source captures the original stty state and restores it before and after child execution. This is the correct architectural place for the responsibility: terminal state belongs to the terminal adapter, not to the text buffer.

Engineering note: A further hardening step is to make restoration robust if resumeTerminal itself fails, and to test abrupt termination and exceptions during child execution.

CHAPTER 10

Input decoding and modal editing

Input reads bytes rather than relying only on line-oriented input. It decodes UTF-8 text and recognizes escape sequences for arrow keys, Home/End, Delete and function keys. This matters because a terminal sends control keys as byte sequences, not as high-level GUI events.

The editor has normal, insert and command modes. Normal mode handles navigation, mode changes, undo/redo, run and save shortcuts. Insert mode inserts text, handles indentation and auto-pairs, and requests autocomplete. Command mode parses editor commands and dispatches file, run, split and settings operations.

Auto-pair behavior

The input layer handles opening delimiters and quotes. It inserts a matching closer and positions the cursor between them. If the expected closer is already at the cursor, typing it moves over the existing character rather than duplicating it. Backspace between an empty bracket pair removes both characters. Enter between an empty brace pair creates an indented block.

Why this belongs in the input layer

Auto-pairs, Tab, Enter and Backspace are user interaction policies. The core TextEditor should provide primitive operations; the input layer composes those primitives into editor behaviors. This keeps the document engine independent of terminal key semantics.

CHAPTER 11

Rendering, invalidation and syntax highlighting

The renderer runs independently from input. It draws visible document lines, cursor, status and UI panels. The README describes an approximately 60 FPS render loop, horizontal and vertical clipping, and per-line highlighter caching. The source includes a dirty-from-line marker in TextEditor so downstream caches can invalidate from the earliest changed line.

A renderer should avoid reconstructing the whole document for every frame. The editor offers getLines(from, to) for a visible range and maintains a line index for row lookup. A render thread can use the dirty marker to know that lines before the first dirty line remain unchanged.

Invalidation contract

TextEditor.takeDirtyFromLine returns the earliest line touched since the previous read. The comment promises that lines before that line are unchanged and that the highlighter state at the start of the dirty line remains valid. This is a useful narrow contract between the editing engine and the renderer.

The renderer and editor must agree on who consumes this marker. The method is documented as intended for a single consumer, the render thread. Adding another consumer could cause it to clear invalidation before the renderer sees it.

Rendering correctness

- A cursor move should not invalidate text-derived caches.
- A text edit should invalidate the changed line and dependent downstream highlighting state.
- A file switch must reset document-specific cache state.
- A resize must recompute viewport dimensions and force a clean redraw.
- Terminal handover must pause rendering so editor output cannot corrupt program output.

CHAPTER 12

File tree, panes and document state

The UI package contains Sidebar, FileIcons, Panes and Split. The README documents a toggleable file tree, keyboard navigation, Nerd Font icons, two-pane splits and commands for opening/reloading files. Panes can share a document when the same path is already open, which avoids two independent buffers silently editing the same file.

The central State class holds shared UI state such as documents, cursor coordinates, mode, focus, autocomplete and status. This makes cross-component access straightforward, but static shared mutable state increases coupling and makes isolated unit tests harder.

A gradual architecture improvement

Do not rewrite the application into a framework just to remove static state. A low-risk step is to introduce a small EditorSession object that owns documents, focus and mode, then pass it to Input, Renderer and Commands. Move one responsibility at a time and preserve behavior with tests.

File icons are a presentation enhancement, not a correctness dependency. The README correctly notes that Nerd Fonts are needed for icon glyphs and provides a setting/environment-variable fallback for terminals without them.

CHAPTER 13

Syntax highlighters and state caching

The syntax package defines a SyntaxHighlighter interface, language-specific C-like and Python highlighters, a plain-text fallback, a registry and a HighlightStateCache. The registry maps file extensions to the appropriate strategy. This is a practical use of the Strategy Pattern: the renderer can ask the selected highlighter to process a line without knowing language-specific rules.

The cache remembers the lexical state at the start of each line. This matters for multi-line constructs such as block comments and triple-quoted Python strings. To highlight a visible line far down the file, the renderer needs the state left by all preceding lines. The cache scans forward in chunks of 512 lines, storing the resulting state for later frames.

Incremental invalidation

If an edit changes line D, the state at the beginning of D can remain valid because it depends only on earlier lines. States after D must be recomputed because the edit may open or close a comment or string that affects later lines. HighlightStateCache.invalidateFrom implements this dependency rule.

This is a good example of a cache with a meaningful invalidation model rather than a time-based cache. Its correctness relies on highlighter state being a sufficient summary of the lexical context at a line boundary.

CHAPTER 14

Autocomplete and Language Server Protocol

Autocomplete first has a local fallback based on words and language keywords. The LSP subsystem provides richer completions and diagnostics when a suitable language server exists on the machine. LspServers maps file extensions to server commands and a language identifier. LspClient launches the server process and speaks JSON-RPC 2.0 over stdin/stdout using Content-Length framing.

LspDocumentSync attaches an EditListener to TextEditor. Mutations enqueue small change records while the editor lock is held; a single scheduled background thread debounces changes for 150 milliseconds and sends one didChange notification. This keeps the input thread from performing network-like or process I/O.

Concurrency and synchronization

The code documents a lock order: editor monitor then sync lock. The background sync thread avoids taking the editor monitor while holding the sync lock. That is the right kind of explicit invariant for preventing deadlocks. Opening a document snapshots current editor content while holding the editor monitor and establishes the sync state under the lock, so an edit is either included in the snapshot or queued after it.

Completion requests are sequence-numbered. When a newer request arrives, older responses are ignored. This prevents stale suggestions from appearing after the user has typed further.

External requirements

LSP is optional and depends on language-server executables installed on the host. A working compiler does not imply a language server is installed. The editor should clearly distinguish 'compiler available', 'language server available' and 'feature unavailable'.

CHAPTER 15

Process execution and compiler planning

CodeRunner separates planning from execution. plan detects the language by extension, resolves the source file, finds a compiler/runtime on PATH and builds a list of command steps. execute launches each step with ProcessBuilder, sets the working directory to the source file's directory and uses inheritIO so the user's program can interact with the real terminal.

Language	Build / check path	Run path
Java	javac with UTF-8, sourcepath and temporary output directory	java -cp temporary-build-dir package.Class
C	gcc, clang or cc; output executable in temp directory; links libm	Execute temporary binary
C++	g++, clang++ or c++; output executable in temp directory	Execute temporary binary
Python	Compile source text for syntax validation without __pycache__	python interpreter on the source file

The Java path reads the package declaration and derives a source root so package-qualified classes can be launched. C and C++ artifacts are written into a temporary build directory. The command list is passed directly to ProcessBuilder, avoiding shell parsing of the user's filename and arguments.

Important safety and correctness boundary

The runner executes the user's program with the user's operating-system permissions. Temporary build directories keep generated artifacts out of the project folder; they do not sandbox the program. A program can still access files and resources permitted to the user. The UI should not imply that execution is isolated or safe for untrusted code.

The SIGINT guard temporarily changes how Ctrl+C is handled so the editor is not accidentally terminated while the child program is running. It must always restore the prior signal handler. This is platform-sensitive code and deserves explicit tests on each supported operating system.

CHAPTER 16

Performance: what the data structures really cost

The performance story has three main layers: text storage, line mapping and rendering. The Piece Table avoids copying the entire document for each edit. The line index avoids rebuilding all line offsets after each edit. The renderer and highlighter cache avoid rescanning and redrawing more than necessary.

Where costs remain

Hot path	Potential cost	How to measure
Piece lookup	Linear scan over piece descriptors	Track piece count; benchmark edits after fragmented workloads
ArrayList mutation	Shifting descriptors on insert/remove	Profile allocation and CPU for random edits
Line index	moveDeltaTo loops and arraycopy on structural edits	Benchmark edits near top/middle/end with and without newlines
getFullText	Copies the full logical document into a StringBuilder/String	Count callers; avoid in per-frame paths
Save ADD pieces	substring creates temporary Strings	Allocation profiler during repeated saves
Highlight catch-up	Scanning all lines between cached frontier and requested line	Open large files and jump to deep lines
LSP full sync	Builds full document text when server requests full sync	Measure memory for large documents

A benchmark must state what it measures. The included JMH benchmark builds a 100,000-line document and defines top/middle insertions, top deletion and sequential typing. Those scenarios are useful, but a benchmark that repeatedly mutates one editor can change the workload over time. Setup scope and invocation behavior should be reviewed so each measurement represents the intended operation.

Engineering note: Do not publish '60 FPS' or 'O(1)' as guarantees without recording machine, JDK, terminal, file size, warmup, measurement and workload.

CHAPTER 17

Memory, resource lifetimes and failure handling

A memory leak is not simply high heap usage. It is memory that remains reachable after the feature or document that needed it has ended. In medit, likely retention surfaces worth auditing include piece fragmentation, the addBuffer, undo/redo action text, per-line highlight states, LSP pending requests and diagnostics, temporary build directories, subprocesses and background executors.

What the source already does well

• Undo history is bounded to 1,500 actions per stack.

• Sequential ADD pieces can coalesce.

• Trailing ADD-buffer text can be released when the final live piece is removed.

• LSP document synchronization uses a single-thread scheduled executor and has an AutoCloseable lifecycle.

• Temporary compiler output is cleaned in a finally block around terminal handover.

• The main renderer is joined briefly and terminal state is restored before LSP shutdown.

What deserves further inspection

The current waitFor(Process) loop catches InterruptedException and continues waiting forever until the child exits. It does not restore the interrupt status. This can make cancellation and shutdown harder to reason about. A better implementation records interruption, continues waiting only if that is the chosen policy, and restores the thread's interrupt flag when the child exits.

The LSP process lifecycle should be tested for server startup failure, server crash, editor close, document switch and a completion response arriving after a document has changed. The source has cleanup logic, but process termination and log-file deletion should be audited end-to-end.

The line-state cache grows with the number of scanned lines. That is a deliberate speed/memory trade-off. If very large files are a target, consider bounded checkpoints or sparse states rather than one byte per line forever.

A useful profiler workflow

• Take a baseline heap snapshot after opening a large file.

• Type, delete, undo and redo repeatedly; take another snapshot.

• Close the document or exit the editor; inspect what remains reachable.

• Look for dominator trees and retained size, not just allocation counts.

• Repeat with LSP disabled and enabled to isolate external-process and cache effects.

CHAPTER 18

Concrete fixes and regression tests

This chapter gives focused corrections grounded in the uploaded code. They are proposals, not modifications already made to your repository.

Fix 1 — Avoid ADD-piece substring allocation while saving

CODE LISTING

```
/* Current pattern */
writer.write(addBuffer.substring(p.start, p.start + p.length));

/* Recommended */
writer.append(addBuffer, p.start, p.start + p.length);
```

StringBuilder implements CharSequence and Writer.append(CharSequence, start, end) writes the requested range without first creating a substring String. Verify output equivalence for empty buffers, mixed ORIGINAL/ADD pieces and non-ASCII text.

Fix 2 — Preserve interruption while waiting for a child process

CODE LISTING

```
private static int waitFor(Process process) {
    boolean interrupted = false;
    try {
        while (true) {
            try {
                return process.waitFor();
            } catch (InterruptedException ex) {
                // The process still owns the terminal. Defer restoring the
                // interrupt flag until waitFor has returned.
                interrupted = true;
            }
        }
    } finally {
        if (interrupted) {
            Thread.currentThread().interrupt();
        }
    }
}
```

This retains the current policy of waiting until the child exits, but it no longer silently discards interruption. A stronger future design could define cancellation explicitly: destroy the child, wait for a grace period, then forcibly terminate if necessary.

Fix 3 — Make save atomic

CODE LISTING

```
Path target = Paths.get(filename).toAbsolutePath();
Path parent = target.getParent();
Path temp = Files.createTempFile(parent, ".medit-", ".tmp");

boolean moved = false;
try {
    try (BufferedWriter writer = Files.newBufferedWriter(
            temp, StandardCharsets.UTF_8)) {
        writePieces(writer);   // stream ORIGINAL and ADD ranges
        if (docLength > 0 && lastChar() != '\\n') writer.write('\\n');
    }
    try {
        Files.move(temp, target,
            StandardCopyOption.REPLACE_EXISTING,
            StandardCopyOption.ATOMIC_MOVE);
    } catch (AtomicMoveNotSupportedException ex) {
        Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING);
    }
    moved = true;
    dirty = false;
} finally {
    if (!moved) Files.deleteIfExists(temp);
}
```

This is a pattern, not a drop-in method: the real implementation must handle a target with no parent, permissions, symlinks, platform-specific atomic-move behavior and errors. Do not mark the buffer clean until the replacement succeeds.

Fix 4 — Add randomized line-index verification

CODE LISTING

```
@Test
void lineIndexMatchesFullScanAfterRandomEdits() {
    Random random = new Random(0x4D56494D);
    TextEditor editor = new TextEditor();

    for (int i = 0; i < 10_000; i++) {
        // Generate a valid insertion or deletion at a random offset.
        // After each operation, compare getLineCount()/getLine()
        // with a reference String model.
        assertDocumentAndLineIndexMatchReference(editor, reference);
    }
}
```

The helper should maintain a simple reference String and apply the same operation to both models. Use a fixed seed for reproducibility, and print the seed and operation sequence on failure.

Fix 5 — Bound history by retained text as well as count

MAX_CAPACITY=1500 bounds action count, but not the size of action.text. A large paste can retain a large String in each history stack. Track the total characters held by both stacks and evict oldest actions until both count and character budgets are satisfied. Keep undo and redo accounting symmetric.

Test matrix

Area	Minimum regression cases
Piece table	Insert at 0/end/middle; delete across pieces; delete all; coalesce; undo/redo
Line index	Multiple newlines; merge lines; empty file; final newline; random edit sequences
Save	Mixed piece sources; UTF-8; write failure; atomic-move fallback; trailing newline policy
Terminal	Normal exit; exception; compile failure; child nonzero exit; terminal restored
LSP	Server unavailable; diagnostics; incremental/full sync; close during pending completion
Unicode	Surrogate pairs; combining marks; wide characters; tab display width

CHAPTER 19

Benchmarking and measurement

The repository includes an EditorBenchmark JMH class under test sources. It creates a 100,000-line text fixture and benchmarks insertion at the top and middle, deletion at the top and sequential typing. JMH is the right kind of tool for JVM microbenchmarks because it manages warmup, measurement iterations and forks.

Benchmark design cautions

• Use independent state per benchmark invocation if operations are destructive and the state drifts.

• Separate setup cost from the measured operation.

• Benchmark both steady-state sequential typing and fragmented random edits.

• Record piece count, document length and line count alongside latency.

• Measure allocation rate and GC, not only average time.

• Benchmark getLine/getLines and rendering preparation separately from edits.

For a useful performance report, include median/average latency, confidence or error estimates, allocation bytes/op, GC activity and the exact JDK and hardware. Compare the same workload before and after each optimization.

Engineering note: A profiler points to a suspected hot path; a benchmark tells you whether a change actually improved it.

CHAPTER 20

The challenges behind medit

The development history discussed in earlier project conversations adds context that source code alone cannot reveal. The project began as a terminal editor and evolved into a more complete programming environment. The name changed from lanterna-editor to medit, short for Maghraby Vim.

The challenges worth preserving

• Owning the editing engine: moving from a basic text representation toward a Piece Table and incremental line index.

• Undo/redo memory: replacing full-document snapshots with small EditAction deltas and a bounded linked stack.

• Cursor correctness: mapping logical row/column positions to linear offsets while preserving cursor behavior through deletion and history replay.

• Terminal handover: making an external program receive the real terminal and restoring editor mode afterward. The stty sane fix was a practical debugging milestone.

• Daily usability: opening and creating files from the current directory with a convenient medit filename workflow.

• Syntax and whitespace: considering syntax coloring, indentation, auto-pairs and the interaction between text and terminal cells.

• Performance investigation: using IntelliJ Profiler and JMH-oriented benchmarks to reason about slow paths and retained memory.

These are not just anecdotes. Each challenge maps to an engineering concept: representation invariants, bounded history, coordinate transforms, resource ownership, command-line integration, lexical state and measurement discipline.

> The most valuable story is that medit became harder as it became more useful. Every new feature introduced another boundary that had to remain correct.

A fair account should also mention the human side without romanticizing overwork: intense project focus can make it difficult to stop, eat regularly, attend classes or take breaks. Sustainable engineering includes pacing, scope control and knowing when to pause. The project's value does not depend on sacrificing health or education.

CHAPTER 21

A practical roadmap

Priority	Work item	Reason / acceptance criteria
P0	Automated randomized core tests	Line index and piece-table invariants survive long edit sequences
P0	Save-path correctness and atomic replacement	No silent data loss; explicit newline and UTF-8 behavior
P0	Process interruption and terminal recovery	Child process exit paths always restore terminal
P1	Unicode display-width model	Cursor and rendering align for tabs, wide glyphs and combining marks
P1	History memory budget	Large paste cannot consume unbounded history memory
P1	LSP lifecycle tests	No stale callbacks or orphan processes after document changes
P2	Encapsulate global State gradually	Reduce coupling while preserving working architecture
P2	Incremental piece index	Improve random edits if profiling proves piece scans dominate
P2	Sparse highlighter checkpoints	Bound cache memory for huge files
P3	Configuration and help documentation	Make settings discoverable and portable
P3	Release automation	Build, test, package and publish reproducible artifacts

Do not optimize blindly

A more complex tree or rope might improve some workloads, but the current Piece Table may be entirely adequate for the target use case. Profile first. The next data structure should solve a measured bottleneck, not merely sound more advanced.

Definition of done

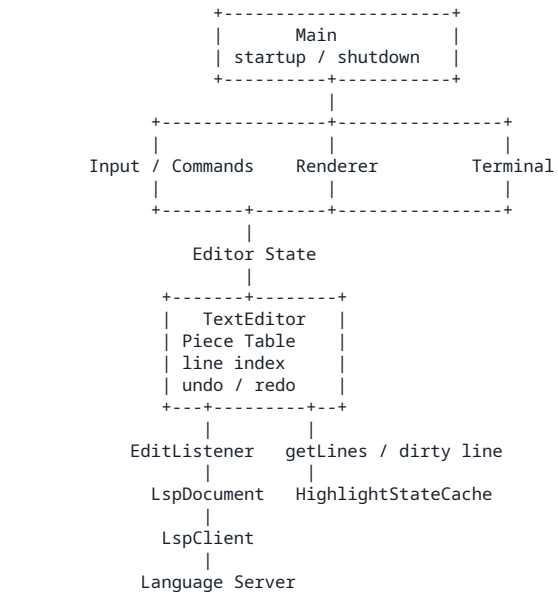
A release-quality milestone should have a reproducible Maven build, automated core tests, a documented JDK requirement, tested terminal restoration, tested runner cleanup, a feature/requirement matrix, and a release artifact built from a tagged commit.

Appendix A — Command and key reference

Mode / command	Behavior documented in README
Normal: i	Enter Insert Mode
Normal: :	Enter Command Mode
Normal: u / r	Undo / Redo
Normal: Ctrl+S / F5	Save / Run or compile
Normal: Ctrl+B	Toggle file tree
Normal: q / Ctrl+Q	Safe quit / force quit
Insert: Esc	Return to Normal Mode
Insert: Tab	Accept suggestion or insert indentation
:e file / :edit file	Open file in focused pane
:e!	Reload / discard unsaved changes
:w [file]	Save buffer
:wq / :x	Save and quit
:q / :q!	Quit safely / force quit
:goto N / :g N / :g $	Jump to line
:run [args]	Build and execute current file
:compile / :build	Compile or syntax-check
:tree / :explorer	Toggle file tree
:set icons / noicons	Toggle file icons
:set autopair / noautopair	Toggle automatic pairs

Appendix B — Architecture map

CODE LISTING




```
Commands -> CodeRunner -> ProcessBuilder -> compiler / runtime
```

This diagram is conceptual. State is currently static and shared; the diagram shows responsibility flow, not a strict dependency-injection graph.

Appendix C — Source review checklist

- Can every edit preserve the invariant: concatenated pieces equal the reference document?
- Does lineStarts match a full scan after every edit type?
- Are offsets, UTF-16 units and terminal display cells kept distinct?
- Does save success mean the bytes have safely replaced the destination?
- Does every terminal handover restore the original terminal settings?
- Can every subprocess and language server be terminated and cleaned up?
- Can callbacks from an old document be rejected after a file switch?
- Are renderer caches invalidated exactly where their dependencies change?
- Are history count and retained-text budgets both bounded?
- Do benchmarks represent stable, reproducible workloads?
- Does README complexity language match actual operations?
- Can a new contributor build, test, run and package the project from a clean checkout?

Closing perspective

medit is a substantial personal engineering project because it crosses the boundaries that make editor software difficult: text representation, terminal I/O, concurrent rendering, external processes, language tooling and stateful user interaction. Its strongest next step is not simply adding more features. It is strengthening the invariants and tests around the features already present.

The book's central lesson is simple: an editor is a collection of cooperating state machines. The buffer, cursor, renderer, terminal, language server and child process each have their own state and lifetime. Reliability comes from making those boundaries explicit, measuring their costs and testing the transitions between them.

انتهى و لله الحمد و المنة و الفضل.